Práctica Grupal UT5 & UT6: roles del equipo y plan de despliegue

Sistema Web de Restaurante: pedidos en línea, cocina y administración, desplegado en la nube (Cloudflare + Vercel + Neon PostgreSQL) bajo un dominio propio.

Dato de la guía	Propuesta del equipo
Nombre del grupo	*(completar)*
Dominio elegido	*(completar; ej.: mirestaurante.com)*: lo registra y configura Javi
Líder de proyecto	**Adolfo Coria**
Enlace repositorio GitHub	*(completar al subir el repositorio)*
Herramienta de gestión	**GitHub Projects** (queda junto al repositorio; sin costo)
Enlace del tablero de tareas	*(completar)*

Los campos en cursiva dependen de decisiones del grupo y deben completarse antes de entregar la guía.

1. Definición del sistema y planteamiento del problema

Nombre del sistema: Sistema Web de Restaurante: pedidos en línea con recojo programado, cocina digital y administración.

A. Árbol de problemas (propuesta)

	Contenido
Efectos (impacto en la empresa y el usuario)	1. Pedidos con errores (ingredientes que el cliente no quería, platos equivocados): reclamos y comida desperdiciada. 2. El cliente espera o recibe la comida fría porque la cocina no sabe a qué hora pasará a recoger o llegará. 3. El dueño no conoce sus ventas reales ni sus platos más vendidos: decide sin datos.
Problema central	La gestión de pedidos del restaurante es manual (teléfono y WhatsApp) y no deja un registro ordenado ni confiable.
Causas raíz (origen técnico u operativo)	1. No existe un canal digital propio para recibir pedidos con todos sus detalles. 2. Pedidos y pagos se anotan a mano, sin estados de preparación ni verificación de los pagos por QR. 3. No hay una base de datos centralizada de menú, clientes y ventas, ni infraestructura en la nube para publicarla.

B. Objetivos (derivados del árbol)

Objetivo	Redacción propuesta
General	Desarrollar y desplegar en la nube, bajo un dominio propio y seguro, un sistema web que gestione los pedidos del restaurante desde el cliente hasta la cocina y la administración.
Específico 1	Implementar un canal web de pedidos que permita quitar ingredientes, elegir para llevar, en mesa o para casa, y programar la hora de recojo o llegada (ataca la causa 1).

Objetivo	Redacción propuesta
Específico 2	Digitalizar el flujo de cocina y de pagos con estados del pedido y verificación de comprobantes QR (causa 2).
Específico 3	Centralizar menú, clientes y ventas en PostgreSQL (Neon) y publicar el sistema con DNS, proxy, SSL/TLS y WAF de Cloudflare sobre Vercel (causa 3).

3. Asignación de roles y responsabilidades (5 integrantes)

Se asignó cada rol oficial de la guía según lo que cada integrante ya trabajó en el proyecto. Javi Mejoia C ocupa el quinto rol, Administrador DNS & Redes, que es el núcleo de la UT5.

Rol / área	Estudiante asignado	Responsabilidades en este proyecto	Firma / V°B°
Líder de Proyecto & DevOps	**Adolfo Coria**	Repositorio GitHub, despliegue continuo en Vercel, variables de entorno, tablero de tareas y coordinación general.	
Backend Developer & DBA	**Eddy Noe Torrico**	Modelo en dbdiagram.io, DDL con Run SQL en Neon, DATABASE_URL, migraciones Prisma, datos iniciales y rutas API.	
Frontend Developer	**Sergio Angulo**	Interfaz (React con Next.js), consumo asíncrono de la API en JSON, CORS, PWA e imágenes en la nube.	
QA & Tester	**Dago Uribe**	Pruebas de endpoints con Postman o Thunder Client, pruebas de carga, validación de transacciones en la base y prueba en navegador.	
Administrador DNS & Redes	**Javi Mejoia C**	Dominio, delegación de nameservers a Cloudflare, registros A y CNAME, proxy, SSL/TLS Full (Strict) y WAF.	

4. Arquitectura de red y topología de servicios

La guía propone frontend en Cloudflare Pages y API en Vercel. Nuestro sistema está hecho en **Next.js**, que sirve la interfaz y la API desde la misma aplicación. Por eso se despliega completo en **Vercel**, y Cloudflare queda delante como DNS, proxy, CDN y WAF:

```
Cliente / navegador web
        |  HTTPS (puerto 443)
        v
Cloudflare  -- DNS del dominio, proxy, CDN, SSL/TLS Full (Strict), WAF
        |  registros A / CNAME hacia Vercel
        v
Vercel (serverless) -- Next.js: páginas del sitio + API REST (/api/...)
        |  DATABASE_URL, SSL (puerto 5432)
        v
Neon -- PostgreSQL serverless (19 tablas)

Archivos: fotos de platos y comprobantes en Cloudinary o Cloudflare R2
```

Consultar con el docente: si exige que el frontend esté separado en Cloudflare Pages, habría que dividir la aplicación en dos proyectos y habilitar CORS en la API. Nuestra propuesta mantiene una sola aplicación en Vercel, más simple y sin CORS, cumpliendo igual el flujo Cloudflare, Vercel y Neon del diagrama.

5. Modelado en dbdiagram.io y ejecución con Run SQL en Neon

Ya están generados a partir del esquema real del sistema, en la carpeta **docs/** del proyecto:

Archivo	Para qué sirve
docs/esquema.dbml	Pegarlo en dbdiagram.io para ver el diagrama: 19 tablas, 26 relaciones (llaves foráneas) y 5 enums.
docs/esquema-postgresql.sql	Script DDL para PostgreSQL. Se ejecuta en Neon, SQL Editor (Run SQL), sobre una base vacía.

Fragmento del DBML (tablas de usuarios y roles):

```
Table roles {
  id text [pk, note: 'cuid']
  code text [unique, not null]     // SUPER_ADMIN | ADMIN | CUSTOMER
  name text [not null]
}
Table users {
  id text [pk, note: 'cuid']
  roleId text [not null]
  email text [unique, not null]
  passwordHash text [not null]
  ...
}
Ref: users.roleId > roles.id
```

Tablas principales

Grupo	Tablas
Usuarios y acceso	roles, users, user_branches, addresses
Menú	categories, products (ingredientes en JSON), branch_products
Pedidos	orders (tipo, hora programada), order_items (precio copiado, ingredientes quitados), order_status_history
Pagos	payments, payment_qrs
Local y soporte	branches, branch_hours, settings, audit_logs (bitácora)
Reservadas	tables (QR por mesa, mejora futura), carts y cart_items (sin uso: el carrito vive en el navegador)

6. Parámetros de DNS y conectividad

Configuración a aplicar en Cloudflare una vez que el dominio y el proyecto de Vercel existan. Los valores exactos de destino los muestra Vercel en *Settings* > *Domains* al agregar el dominio: **usar siempre los que indique Vercel**.

Parámetro	Valor	Proxy	Para qué
Nameservers	Los 2 que asigne Cloudflare (ej.: *xxx.ns.cloudflare.com*)	-	Se cambian en el registrador del dominio para delegarlo a Cloudflare
A @	76.76.21.21 (IP de Vercel para el dominio raíz)	Activado	mirestaurante.com apunta a Vercel
CNAME www	cname.vercel-dns.com	Activado	www.mirestaurante.com apunta a Vercel
SSL/TLS	Full (Strict)	-	Cifrado de punta a punta con certificado válido en Vercel
Edge Certificates	Always Use HTTPS + HSTS	-	Fuerza HTTPS en todo el sitio
WAF	Reglas administradas + límite de solicitudes en /login y /api/orders	-	Frena ataques y abusos (fuerza bruta, pedidos masivos)
Variables en Vercel	DATABASE_URL (Neon, sslmode=require) y AUTH_SECRET	-	Conexión segura a la base y firma de sesiones

Qué hay que cambiar en el proyecto para cumplir la guía

Hoy el sistema corre en local con MySQL (XAMPP). Para la práctica hay que llevarlo a la nube. Orden sugerido:

#	Tarea	Responsable
1	Subir el proyecto a GitHub (hacer commit de los cambios pendientes) y crear el tablero en GitHub Projects.	Adolfo
2	Crear el proyecto en Neon, revisar el diagrama en dbdiagram.io y ejecutar docs/esquema-postgresql.sql con Run SQL.	Eddy
3	Cambiar Prisma a PostgreSQL (provider = "postgresql"), crear la migración inicial y marcarla como aplicada en Neon (npx prisma migrate resolve --applied <nombre>). Cargar los datos iniciales (npm run db:seed).	Eddy
4	Mover la subida de imágenes y comprobantes a Cloudinary o Cloudflare R2: Vercel no guarda archivos en disco.	Sergio + Eddy
5	Importar el repositorio en Vercel, cargar DATABASE_URL y AUTH_SECRET y verificar el despliegue automático.	Adolfo
6	Registrar el dominio, delegarlo a Cloudflare y crear los registros A y CNAME, con SSL Full (Strict) y WAF.	Javi
7	Probar endpoints, carga y transacciones sobre el sitio publicado; reportar fallos en el tablero.	Dago
8	Cambiar las contraseñas de prueba y el número de WhatsApp si corresponde, antes de la entrega.	Adolfo

Líder de Proyecto & DevOps

Adolfo Coria

Misión: que el proyecto esté versionado, desplegado de forma automática y que el equipo cumpla los plazos.

Tareas concretas

- □ Crear el repositorio en GitHub, subir el proyecto y proteger la rama *main*.
- □ Crear el tablero en GitHub Projects con las tareas del plan y asignarlas a cada integrante.
- □ Conectar el repositorio con Vercel: cada *push* a *main* despliega a producción y cada *pull request* crea una vista previa (CI/CD).
- □ Opcional: GitHub Actions que ejecute *npm run lint* y la verificación de tipos en cada pull request.
- □ Cargar en Vercel las variables DATABASE_URL y AUTH_SECRET (nunca subirlas al repositorio: .env está en .gitignore).
- □ Coordinar la demo final y presentar la apertura y el cierre.

Debe saber explicar

- El problema, los objetivos y el alcance (árbol de problemas).
- Qué es CI/CD y cómo Vercel despliega solo al recibir un push.
- Por qué los secretos van en variables de entorno y no en el código.
- Las decisiones de producto: se quitó el QR por mesa en esta etapa por seguridad; «En mesa» quedó como pedido con hora de llegada.

Preguntas probables del docente

¿Cómo llega un cambio del código a producción?

Se hace commit y push a main en GitHub; Vercel detecta el cambio, compila y publica la nueva versión en minutos. Si falla la compilación, se queda la versión anterior.

¿Dónde están las contraseñas de la base de datos?

En variables de entorno de Vercel. En el repositorio solo hay un .env.example sin datos reales.

Backend Developer & DBA

Eddy Noe Torrico

Misión: que la base de datos esté bien modelada, creada en Neon y conectada de forma segura.

Tareas concretas

- □ Pegar docs/esquema.dbml en dbdiagram.io, revisar llaves foráneas y cardinalidades, y exportar la imagen del diagrama.
- □ Crear el proyecto en Neon y ejecutar docs/esquema-postgresql.sql en el SQL Editor (Run SQL).
- □ Copiar la cadena de conexión de Neon (con sslmode=require) y entregarla a Adolfo para Vercel como DATABASE_URL.
- □ Cambiar Prisma a PostgreSQL, alinear el historial de migraciones con la base de Neon y ejecutar el seed.
- □ Documentar las rutas API: POST /api/orders, GET /api/admin/orders, GET /api/superadmin/clientes.csv, GET /api/files/{nombre}, GET /api/comprobantes/{nombre}.

Debe saber explicar

- Las 19 tablas y sus relaciones; por qué order_items copia nombre y precio (el historial no cambia si cambia el menú).
- Que los precios se recalculan en el servidor y que un pedido se guarda en una sola operación atómica (pedido, líneas, historial y pago).
- Qué es un ORM (Prisma) y por qué evita la inyección SQL (consultas parametrizadas).
- Qué es Neon: PostgreSQL serverless, conexión cifrada por SSL en el puerto 5432.

Preguntas probables del docente

¿Qué cardinalidad tiene un pedido con sus platos?

Uno a muchos: un pedido (orders) tiene muchas líneas (order_items); cada línea pertenece a un solo pedido.

¿Por qué los ID son texto y no serial?

Prisma genera identificadores cuid: son únicos sin depender de la base y no revelan cuántos registros hay.

¿Cómo se conecta la API a Neon?

Con la variable DATABASE_URL, que incluye usuario, contraseña, host de Neon y sslmode=require para cifrar la conexión.

Frontend Developer

Sergio Angulo

Misión: que la interfaz funcione bien en celular y computadora y consuma la API de forma correcta.

Tareas concretas

- □ Explicar las pantallas: menú, ficha del plato con ingredientes, carrito, checkout (para llevar, en mesa, para casa y hora), seguimiento y QR del pedido.
- □ Mostrar el consumo asíncrono: el checkout envía el pedido con fetch en JSON a POST /api/orders; el tablero de cocina consulta GET /api/admin/orders cada 5 segundos.
- □ Cambiar la subida de imágenes a Cloudinary o R2 junto con Eddy (único archivo a modificar: src/lib/upload.ts).
- □ Verificar la instalación como app (PWA) sobre el dominio con HTTPS.
- □ Revisar que el sitio se vea bien en celular antes de la entrega.

Debe saber explicar

- React con Next.js: páginas generadas en el servidor y componentes interactivos en el navegador.
- Qué es CORS y por qué aquí no hace falta: la interfaz y la API están en el mismo dominio. Si el frontend se separara en Cloudflare Pages, la API tendría que permitir ese origen.
- Qué es una PWA y por qué necesita HTTPS.
- Cómo se guarda el carrito en el navegador y por qué el servidor no confía en los precios que envía.

Preguntas probables del docente

¿Qué formato usan para comunicarse con la API?

JSON. Por ejemplo, el pedido se envía como un objeto con los productos, cantidades, ingredientes quitados, tipo y hora.

¿Qué es CORS?

Una regla del navegador que bloquea peticiones a otro dominio salvo que el servidor lo permita. Como servimos todo desde el mismo dominio, no se activa.

QA & Tester

Dago Uribe

Misión: demostrar con evidencia que el sistema funciona y aguanta la carga esperada.

Tareas concretas

- □ Armar una colección en Postman o Thunder Client con los endpoints (para los que requieren sesión, iniciar sesión en el navegador y copiar la cookie *session*).
- □ Casos mínimos: crear pedido válido (200), sin sesión (400), hora no disponible (400), quitar un ingrediente fijo (400); tablero sin permiso (401).
- □ Prueba de carga a la página del menú y al tablero (por ejemplo con k6 o autocannon) y registrar la latencia promedio.
- □ Validar transacciones en Neon: tras un pedido, verificar que existan sus filas en orders, order_items, order_status_history y payments.
- □ Repetir la prueba en navegador de punta a punta (cliente, trabajador y dueño) sobre el sitio publicado.
- □ Registrar cada fallo como tarea en el tablero y verificar la corrección (verificación cruzada).

Debe saber explicar

- Diferencia entre prueba funcional, de carga y de punta a punta.
- La prueba en navegador ya realizada en local: 33 comprobaciones, todas correctas.
- Los códigos HTTP que devuelve la API (200, 400, 401, 404, 500) y qué significa cada uno.
- Las validaciones de seguridad que se probaron: horas inválidas, ingredientes fijos, cuentas desactivadas.

Preguntas probables del docente

¿Cómo prueban que un pedido se guarda completo?

Se crea por la API y luego se consulta en Neon que existan el pedido, sus líneas, su historial y su pago. Se guardan en una sola operación: o se guarda todo o nada.

¿Qué latencia es aceptable?

Para un restaurante pequeño, respuestas por debajo de un segundo con decenas de usuarios simultáneos. Se mide y se reporta con la herramienta de carga.

Administrador DNS & Redes

Javi Mejoia C

Misión: que el sistema sea accesible por el dominio propio, con HTTPS de punta a punta y protegido por Cloudflare.

Tareas concretas

- □ Registrar o elegir el dominio y crear la cuenta de Cloudflare.
- □ Agregar el dominio en Cloudflare y cambiar los nameservers en el registrador por los dos que indica Cloudflare (delegación).
- □ Crear los registros A (@) y CNAME (www) hacia Vercel con el proxy activado (nube naranja).
- □ Poner SSL/TLS en Full (Strict), activar Always Use HTTPS y HSTS.
- □ Configurar el WAF: reglas administradas y límite de solicitudes para /login y /api/orders.
- □ Verificar con nslookup, dig y curl -I (ver la cabecera *server: cloudflare*) y documentar con capturas.

Debe saber explicar

- Qué es DNS y cómo se resuelve un nombre: registrador, nameservers, registros A y CNAME, TTL y propagación.
- Qué hace el proxy inverso de Cloudflare: oculta la IP real, sirve como CDN y aplica el WAF.
- Diferencia entre los modos SSL Flexible, Full y Full (Strict), y por qué Strict es el seguro.
- Qué es un WAF y qué ataques frena (inyección, bots, fuerza bruta).

Preguntas probables del docente

¿Por qué Full (Strict) y no Flexible?

En Flexible el tramo entre Cloudflare y el servidor viaja sin cifrar. En Full (Strict) va cifrado y Cloudflare además valida que el certificado de Vercel sea auténtico.

¿Qué diferencia hay entre un registro A y un CNAME?

El registro A apunta un nombre a una dirección IP; el CNAME apunta un nombre a otro nombre (por ejemplo, www a cname.vercel-dns.com).

¿Cómo comprueban que el tráfico pasa por Cloudflare?

Con curl -I al dominio: la respuesta trae la cabecera server: cloudflare y un identificador cf-ray. nslookup devuelve IP de Cloudflare, no la de Vercel.

Lo que los cinco deben saber

El docente puede preguntar a cualquiera. Estos puntos los debe poder explicar cualquier integrante:

- **Qué hace el sistema:** el cliente pide (quitando ingredientes, eligiendo para llevar, en mesa o para casa y la hora), paga en efectivo o QR y recibe un QR de su pedido para enviar por WhatsApp; la cocina lo atiende en un tablero; el dueño administra y ve ventas.
- **El recorrido de una petición:** navegador, Cloudflare (DNS, proxy, WAF), Vercel (Next.js), Neon (PostgreSQL), y la respuesta de vuelta por HTTPS.
- **Seguridad:** contraseñas con bcrypt, sesión en cookie httpOnly firmada, permisos por rol verificados en el servidor, precios recalculados en el servidor, HTTPS de punta a punta.
- **Quién hizo qué:** cada uno debe poder decir en una frase el rol de los otros cuatro.

Cuentas de prueba (cambiarlas antes de publicar)

Perfil	Correo	Contraseña
Cliente	cliente@demo.com	Cliente123!
Trabajador	trabajador@restaurante.local	Trabajo123!
Dueño (Super Admin)	superadmin@restaurante.local	Admin123!

Limitaciones que conviene reconocer

- El tablero de cocina se actualiza cada 5 segundos (no es instantáneo); con más volumen se usarían WebSockets.
- La verificación del pago QR es manual: los bancos no ofrecen una API abierta para confirmarlo.
- Una sola sucursal activa, aunque la base de datos ya soporta varias.
- El QR por mesa se dejó para una etapa futura.